



一手交换五子棋游戏编程对抗赛

在本题中，你的任务是编写五子棋 AI 程序，即让程序自动下棋，并在北京大学 Botzone 平台 (<https://www.botzone.org.cn/>) 与其他同学对战；还将进行计算机学院、软件学院跨院 PK，决出两院总冠军。

一、游戏规则与术语：

1. 棋盘为 15×15 围棋棋盘。
2. 黑子先手，双方轮流落子，落子后棋子不可移动。
3. 本方 5 子横、竖、斜连成一线，则获胜。

4. 通过研究发现，五子棋存在平衡点，所以本题采用“**一手交换**”规则，能够达到限制五子棋先手优势的作用。（迫使黑走平衡开局，否则交换后劣势）。具体做法是：在黑棋第一手后（棋盘上有黑一个棋子），白棋可以选择是否换手。如果换手，则黑棋变白、白棋变黑，换手后下一手由黑棋下。

具体规则参见：<https://wiki.botzone.org.cn/index.php?title=Gomoku-Swap1>

每步走棋限时 1 秒，内存限定 256MB。任何非法操作会被立即判负，如程序崩溃、超时、坐标越界、格式错误等。

建议大家实际玩玩这两款游戏，有利于掌握其规则及玩法。大家也可以查阅相关网络资料，进一步了解其技巧和经验。

大家无需掌握图形界面编程技术，只需将核心代码上传至北京大学 Botzone 对战平台 (<https://www.botzone.org.cn/>)，对战平台将负责在线对战和图形界面展示，并给出排名。本题编程语言为 C/C++ 语言，允许使用 C++ STL。

二、代码实现：

大家无需掌握图形界面编程技术，只需将代码上传至北京大学 Botzone 对战平台 (<https://www.botzone.org.cn/>)，对战平台将自动进行图形化对战比赛，并给出排名。本题编程语言为 C/C++ 语言，允许使用 C++ STL。

（1）棋盘坐标设定

15×15 的二维棋盘分为横轴和纵轴两个维度，以左上角为坐标原点 (0, 0)。坐标系如图 1 所示。在程序中棋盘信息存储在数组 `int board[15][15]` 中，下标从 0 开始，元素 `board[x][y]` 有 0、1、-1 三种可能取值，分别表示棋盘 (x, y) 处为空白（没有任何棋子）、对方棋子、我方棋子。

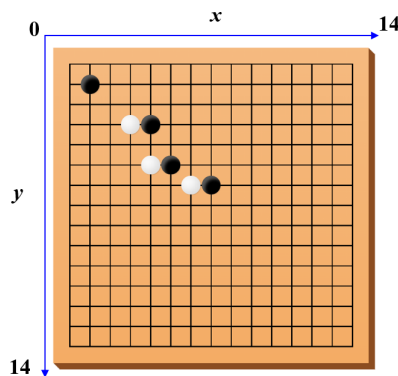


图 1 棋盘坐标系

(2) 你的程序与对战平台的交互方式

本题提供给大家**样例程序**，样例程序已经包含了程序与对战平台的交互功能，你只需在样例代码基础上开发，基于当前棋局，输出本回合的决策结果。你的程序（在botzone网站里也称为bot）的生命周期仅为一次决策：读入当前棋盘状态，输出本回合你的下棋（落子）位置。在整个对弈过程中，对战平台会多次调用你的程序（每下一步棋调用你一次，直至对弈结束）。

对战平台每次调用你的程序时：首先将之前对方和本方落子的位置作为输入传送给你的程序，从而建立起棋盘信息board数组，然后你的程序根据棋盘信息进行决策，并向对战平台输出你的决策结果，即本方的落子位置。

你的bot与对战平台的交互方式：你的程序通过一系列数字与对战平台进行交互，格式形如下表左侧所示。而你的程序就是要读取这些数字，建立棋盘信息，并基于棋盘信息做决策，最后输出第 k 回合本方落子位置。若我方是先手，则第一回合读入的对方落子位置是(-1,-1)。详见示例程序。例如图2的棋盘状态对应下表右侧的输入。

k	//双方已下棋的回合数	4
$x_1^{\text{对方}} y_1^{\text{对方}}$	//第1回合对方落子位置	-1 -1
$x_1^{\text{本方}} y_1^{\text{本方}}$	//第1回合本方落子位置	2 3
$x_2^{\text{对方}} y_2^{\text{对方}}$	//第2回合对方落子位置	0 8
$x_2^{\text{本方}} y_2^{\text{本方}}$	//第2回合本方落子位置	7 5
.....		6 8
$x_{k-1}^{\text{对方}} y_{k-1}^{\text{对方}}$	//第k-1回合对方落子位置	5 8
$x_{k-1}^{\text{本方}} y_{k-1}^{\text{本方}}$	//第k-1回合本方落子位置	1 4
$x_k^{\text{对方}} y_k^{\text{对方}}$	//第k回合对方落子位置	

假设本方是黑方，对于上面的信息，可知之前的棋应该是这么下的：我方第一回合收到(-1,-1)，然后我方下棋(2,3)，然后对方下棋(0,8)，然后我方下棋(7,5)...以此类推，形成图2的棋盘。

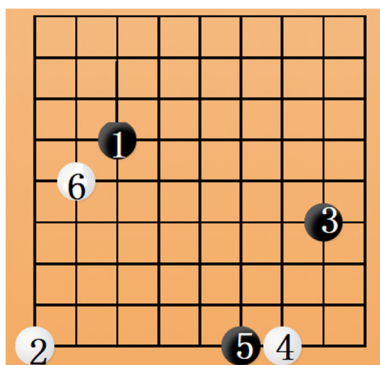


图2 对战平台与博弈程序通信示例

(3) 博弈程序编写

提供给大家样例程序，为Gomoku_simple.cpp，里面包含了详细注释，容易阅读。代码中棋局初始化和输入输出操作你都不用管，**你只需要实现代码中下图红框部分，用你的策略替换掉红框中的样例策略。**new_x和new_y就是你的决策结果。**如果我方是后手，第一回合输出(-1,-1)表示换手。**

```

1  cin >> x >> y;
2  if (x != -1) board[x][y] = -1; //对方
3
4  //此时board[][]里存储的就是当前棋盘的所有棋子信息，x和y存的是对方最近一步下的棋
5
6  /*****在下面填充你的代码，决策结果（本方将落子的位置）存入new_x和new_y中*****/
7  //下面仅为随机策略的示例代码，可删除
8  int avail_x[SIZE*SIZE], avail_y[SIZE*SIZE]; //合法位置表
9  int cnt = 0;
10 for (int i = 0; i < SIZE; i++)
11     for (int j = 0; j < SIZE; j++)
12         if (board[i][j] == 0) {
13             avail_x[cnt] = i;
14             avail_y[cnt] = j;
15             cnt++;
16         }
17 srand(time(0));
18 int rand_pos = rand() % cnt;
19 int new_x = avail_x[rand_pos];
20 int new_y = avail_y[rand_pos];
21 /*****在上方填充你的代码，决策结果（本方将落子的位置）存入new_x和new_y中*****/
22
23 // 向平台输出决策结果
24 cout << new_x << ' ' << new_y << endl;
25 return 0;
26

```

(4) 博弈程序的调试

➤ 本地调试

在本地编译运行程序，本地调试时可采用“简单交互”方式，并结合输入输出重定向，以数字形式作为输入，并观察程序输出结果。

➤ 在Botzone网站上在线调试

Botzone 提供了比较完善的调试功能。点击网页右上角的“我的 Bot”，然后点击下方“创建新 Bot”，如图 3。然后填写 bot 信息，上传代码文件，如图 4，。请不要勾选“开源”，为避免抄袭，将对开源者扣分。**Botzone 提供了较好的版本管理功能，方便管理程序的各个版本。**若想对已有的 bot 增加新的版本，可如图 5 所示点击加号。



图 3 创建新 Bot



图 5 为已有 bot 增加新版本



图 4 配置 Bot 信息上传代码



图 6 创建游戏桌

在线调试，可回到首页，手动创建游戏桌，如图 6 所示。然后指定玩家进行对局，如图 11 所示，对局双方可以是已有的 Bot，也可以是房主（人类），即你自己。



图 7 指定玩家进行对局调试

在对局页面（也可通过右上角菜单“我的对局”选择对局点击“回放”）在线调试，点击下方“切换到调试模式”，如图 9 所示。详细调试信息可以点击右下角“log 查看工具（调试用）”查看每回合每方具体的输入和输出信息，如图 8，选择“查看完整 Log”，也可以查看

程序的运行时间。为防止程序超时，可以尝试进行卡时，比如在每次迭代/循环时，通过 `clock()` 或者 `time()` 函数来判断程序运行了多久，从而决定是否提前返回。

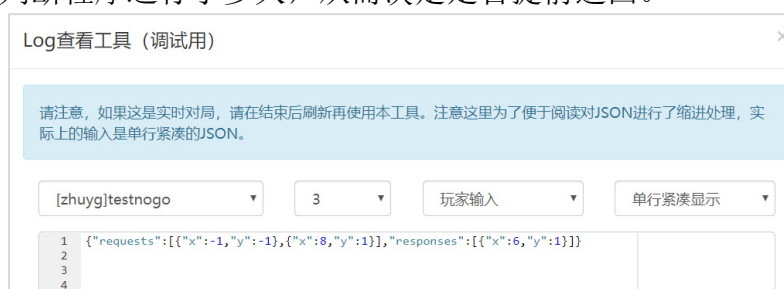


图 8

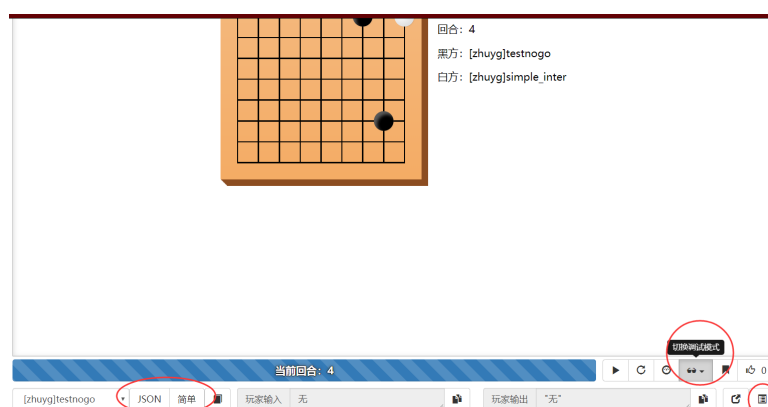


图 9

额外提示 1：你的程序在本地的输出和平台的输出不一样怎么办？（1）你的程序可能存在漏洞，在平台上可能会暴露出数组越界、地址越界、除零、未初始化等问题。建议仔细检查自己的代码。TScanCode 是一个很好的检查代码漏洞的工具。（2）如果采用了随机数，可能你本地生成的随机数和平台生成的随机数不一致。

额外提示 2：关于超时和卡时

在比赛的时候，bot 程序会被 Botzone 调度到一个评测机集群上运行，评测机是两台物理机上的多个虚拟机，当平台负载比较高的时候，物理机会存在总线竞争之类的问题，导致虚拟机内的运行速度变慢，比如原本 bot 一步决策花 0.5s，可能平台高负载的时候就要花 0.9s。可能导致个别同学平时自己创建游戏桌单局对弈时都不会超时，但在比赛中就被判超时。

因此建议大家在 bot 程序中加入卡时的代码，比如程序中有个耗时循环（如 MCTS 迭代次数）的话，不要固定循环次数，而是在每次循环的时候通过 `clock()` 函数获取当前程序已运行的时间，当程序运行接近 1 秒时就退出循环，这样即使运行速度变慢，也不会超时，只是循环次数变少了而已。例如：

```
int threshold = 0.95 * (double)CLOCKS_PER_SEC; //CLOCKS_PER_SEC 和 clock() 的含义可百度
int start_time, current_time;
start_time = current_time = clock();
while (current_time - start_time < threshold){ //到 0.95 秒立即跳出循环
    monte carlo tree search.....
    current_time = clock();
}
```